Week 3: Searching Smarter, Not Harder

Greedy Best-First Search

FIT1061: Introduction to Artificial Intelligence

2026-07-27

The Story: AlphaGo's Move 37

On 13 March 2016, a program called **AlphaGo** sat down (through a human assistant who placed its stones) against **Lee Sedol**, one of the strongest Go players alive. Midway through the second game of their five-game match, AlphaGo played its **thirty-seventh move**. The professional commentators watching thought it was a mistake. One of them said, out loud, that a human would never play there. Lee Sedol left the room to compose himself. Then the game continued, and AlphaGo won.

To feel why that move mattered (and why it is the perfect way into this week) you first have to understand why Go had resisted computers for so long.

Why Go is so much harder than chess

Nineteen years earlier, IBM's Deep Blue had beaten the world chess champion by sheer search speed: it looked at roughly **200 million positions every second** and searched about six moves ahead. That worked for chess because a chess position offers only about **35** legal moves, so the tree of possibilities, while enormous, could be cut down and scored with hand-written rules.

Go breaks that approach twice over. A Go board has **361** points and offers around **250** legal moves at each turn, and a game runs for about **150** moves. The number of possible games is larger than the number of atoms in the observable universe: so large that no computer, however fast, can search even a tiny fraction of it. That is the first wall: Go is **too big to search blindly**.

The second wall is subtler. In chess, you can write down decent rules for judging a position: count the pieces, check whether the king is safe. In Go, the value of a stone depends on patterns that only pay off fifty moves later. Strong players describe good moves as a matter of *feel*,

not calculation. So Go was not only too big to search; it was also **too subtle to score** with hand-written rules.

The move no one had programmed

AlphaGo got past both walls the same way you get past a maze you cannot see all of at once: it used **hints**. Rather than examine all 250 moves at every turn, it leaned on a learned sense of *which handful of moves were even worth considering*, and a learned sense of *how good a position looked* without playing it out to the end. Those hints let it spend its limited search where it mattered.

Move 37 was the payoff. It was not a move any human textbook recommended, and it was not the result of exhaustively checking every reply. It came from a hint (an estimate of what looked promising) that happened to point somewhere no human intuition had. The machine did not understand Go the way Lee Sedol did. It searched, but it searched **guided by an estimate** rather than blindly.

That single idea (*let an estimate steer the search*) is what this week gives you the tools to build. The estimate has a name, **the heuristic**, and the simplest search that trusts one is the subject of everything that follows: **greedy best-first search**.

> **i** Further viewing
>
> *AlphaGo: The Movie* (2017) follows the full Lee Sedol match, including Move 37 and the human reaction to it. DeepMind has posted it free: watch on YouTube (90 min).

> Going deeper — optional, beyond this unit, not assessed
>
> Curious how AlphaGo was actually *built*? The *Dwarkesh Podcast* episode **Building AlphaGo from scratch** (with Eric Jang, who rebuilt it for under $10,000) walks through the training recipe: Monte Carlo tree search, self-play, and reinforcement learning. This is graduate-level material that goes well past anything in this unit — **none of it is examinable**. Treat it as a glimpse of where these ideas lead. Watch on YouTube.

The Problem: Search With a Sense of Direction

Last week's searches were **blind**. Breadth-first search fanned out in rings from the start, one layer at a time, treating every direction as equally worth exploring. It always found the shortest route, but it paid for that guarantee by exploring almost everything in reach before stumbling on the goal.

Picture a delivery drone crossing a city grid from one corner to the opposite corner. Breadth-first search would sweep outward in every direction (north, south, east, west) colouring in more and more of the map until the goal finally fell inside the expanding circle. But you already know something the blind search ignores: the goal is to the **south-east**. Every step north is almost certainly wasted. A search that *knew which way the goal lay* could pour its effort into the promising direction and leave the rest of the map untouched.

That "sense of direction" is exactly what a **heuristic** provides.

> **i** Definition: heuristic
>
> A **heuristic** is a practical rule of thumb for making a good decision *without checking every option*. In our searches, we write it as $h(n)$: an **estimate of the remaining cost** from a node n to the goal. It does not have to be exact; it only has to be useful.

On a grid, two heuristics come up again and again. Suppose our current cell and the goal differ by Δr rows and Δc columns.

- **Manhattan distance** counts grid steps, as if you could only walk along streets: $h_M = |\Delta r| + |\Delta c|$. It fits a world where you move up, down, left, and right, never diagonally.
- **Euclidean distance** is the straight-line, as-the-crow-flies gap: $h_E = \sqrt{(\Delta r)^2 + (\Delta c)^2}$. It fits a world with open space and free movement.

Both are cheap to compute and neither peeks at the actual walls or roads; they are only *guesses* at how far is left. On a four-direction grid, Manhattan is usually the better guess, because it counts the steps you are actually allowed to take.

Heuristic	Formula	Answers the question
Manhattan	$\lvert\Delta r\rvert + \lvert\Delta c\rvert$	How many grid steps, moving in four directions?
Euclidean	$\sqrt{(\Delta r)^2 + (\Delta c)^2}$	How far in a straight line?

The heuristic is the whole point of this week. Once we have a number that estimates *how far is left*, we can build a search that always walks toward the smaller number.

The Algorithm: Greedy Best-First Search

Here is the pleasant surprise. To turn a blind search into a guided one, you change **one thing**.

Breadth-first search kept its list of places-to-explore (the **frontier**) in a plain queue, serving them strictly in the order they arrived (first in, first out). Greedy best-first search keeps the

frontier in a **priority queue** instead: a waiting list that always hands back the item with the *smallest priority value*, no matter when it arrived. We set each node's priority to its heuristic $h(n)$. The effect is that the search always expands, next, whichever waiting node *looks* closest to the goal.

> **i** Definition: priority queue
>
> A **priority queue** is a waiting list where each item carries a **priority number**. Instead of serving items in arrival order, it always releases the item with the **lowest priority value** first. (Ties are broken by arrival order.) Swapping a plain queue for a priority queue (and using $h(n)$ as the priority) is the only structural change from last week's search.

The procedure reads almost identically to breadth-first search:

1. Start with the frontier holding only the **start** node, given priority h(start).
2. Repeat: take from the frontier the node with the **lowest** priority; call it the **current** node.
3. If the current node **is the goal**, stop; we have arrived.
4. Otherwise, look at each **neighbour** of the current node. For any neighbour we have not already recorded, add it to the frontier with priority h(neighbour), and remember that we reached it *from* the current node.
5. If the frontier ever empties without reaching the goal, there is no path.

Step 4's "remember where we came from" is what lets us **rebuild the path** at the end: once we pop the goal, we walk the "came-from" links backwards, from goal to start, and reverse them. Everything else (the frontier, the neighbour lookup, the visited check) carries straight over from last week. The single new ingredient is that the priority is $h(n)$, the estimate of what lies ahead, rather than plain arrival order.

A worked example

Let us trace it on a small graph. Five nodes: **S** (start), **A**, **B**, **C**, and **G** (goal). They are joined by these roads, each road one step long:

From	Connects to
S	A, B
A	G
B	C
C	G

Someone has handed us a heuristic: an estimate of the remaining distance to **G** from each node:

Node	S	A	B	C	G
$h(n)$	7	6	2	1	0

We will **trust the hint** and always expand the frontier node with the lowest h. Watch what happens.

Step	Pop (lowest h)	Newly added to frontier (with h)	Frontier after
1	**S** (h=7)	A (6), B (2)	B(2), A(6)
2	**B** (h=2)	C (1)	C(1), A(6)
3	**C** (h=1)	G (0)	G(0), A(6)
4	**G** (h=0)	goal reached	

Following the "came-from" links back from **G**, greedy returns the path

$$\text{S} \to \text{B} \to \text{C} \to \text{G} \qquad (\textbf{3 steps}).$$

Greedy never spent any time on A: it sat untouched in the frontier the whole time. The heuristic did real work at the very first step, steering us toward B (estimate 2) and away from A (estimate 6).

Why Greedy Is Fast but Not Optimal

Now look again at that graph, and find the route through **A**:

$$\text{S} \to \text{A} \to \text{G} \qquad (\textbf{2 steps}).$$

The path greedy found is **longer than the shortest one** — three steps instead of two. Greedy walked S → B → C → G when a two-step route was sitting right there through A.

What went wrong? At the very first step, node A carried a heuristic of 6 (nearly as high as the start itself) while node B looked far more promising at 2. Greedy trusted the estimate, chased B, and by the time the goal was in hand it had never come back to try the "unpromising" A. But A was the doorway to the shortest path. **The hint was misleading, and greedy had no way to know.**

This is the essential flaw, and it is worth stating plainly:

! Key idea: greedy is not optimal

Greedy best-first search only ever looks at $h(n)$: its **guess of what lies ahead**. It never accounts for **how far it has already travelled**. So it can be lured down a route that looks close at every step yet adds up to a long journey. Greedy is fast and often right, but when it is wrong, it is **wrong about path length**: it returns *a* path, not necessarily the *shortest* one.

The trade-off against last week's blind search is now clear:

	Breadth-first (blind)	Greedy best-first (guided)
Waiting list	Plain queue (first in, first out)	Priority queue, ordered by $h(n)$
Uses a heuristic?	No	Yes
Shortest path?	**Always**	Not guaranteed
Speed	Slow: explores widely	Fast: explores toward the goal

On most maps, greedy is a bargain: it finds the same good route as breadth-first while exploring a fraction of the cells. But hand it a map with a wall or a detour that the straight-line estimate cannot see, and it will confidently take the longer way round.

That raises an obvious question. Greedy ignored the distance it had *already* paid; what if a search accounted for **both** the cost so far *and* the estimate of what remains? That search exists, it fixes exactly this flaw, and it is waiting for you in short supplementary notes for anyone who wants to go further.

i Further viewing

Veritasium's *How Does Google Maps Actually Work?* traces this week's core idea — *let an estimate steer the search* — from Dijkstra's original algorithm to the search that accounts for both costs at once (its heuristic is the straight-line distance you met above), and on to the tricks that route a whole continent in microseconds. It also shows why the *shortest* path is not always the *fastest* one: real roads carry different costs — the edge **weights** we build on next. Watch on YouTube (30 min).

Summary

Key takeaways:

- **Blind search wastes effort.** Breadth-first search explores in every direction equally. It is guaranteed to find the shortest path, but it explores far more of the map than it needs to.
- **A heuristic is a sense of direction.** The heuristic $h(n)$ estimates the remaining cost from a node to the goal. Manhattan distance counts grid steps; Euclidean distance measures the straight line. Neither is exact; each is a useful guess.
- **Greedy best-first search is one swap from breadth-first search.** Replace the plain queue with a priority queue ordered by $h(n)$, and the search always expands whichever waiting node *looks* closest to the goal.
- **Greedy is fast.** By pouring effort toward the goal instead of sweeping outward, it typically explores a fraction of the cells a blind search would.
- **Greedy is not optimal.** Because it looks only at the estimate ahead and ignores the distance already travelled, a misleading heuristic can send it down a longer route. It returns *a* path, not always the *shortest*.

Key vocabulary:

Term	Meaning
Heuristic $h(n)$	An estimate of the remaining cost from a node to the goal; a rule of thumb, not an exact figure
Manhattan distance	Grid-step distance, $\lvert\Delta r\rvert + \lvert\Delta c\rvert$, for movement in four directions
Euclidean distance	Straight-line distance, $\sqrt{(\Delta r)^2 + (\Delta c)^2}$
Priority queue	A waiting list that releases the item with the lowest priority value first, regardless of arrival order
Frontier	The set of discovered-but-not-yet-explored nodes waiting to be expanded
Greedy best-first search	A search that expands whichever frontier node has the lowest heuristic value
Optimal	Guaranteed to return a shortest path; greedy best-first search is *not* optimal

What's next: greedy trusts the hint blindly. Next week we ask what happens when there is no goal to walk toward at all: only a candidate we try to improve, one small step at a time.

In the Lab

This week: give a search a sense of direction, then watch where the hint helps and where it misleads.

You will build greedy best-first search from the parts you already have, and put it head-to-head with last week's blind search.

1. **Compute the two heuristics.** Write the Manhattan and Euclidean distance between two grid cells: the two estimates of "how far is left" that will drive the search.
2. **Build a priority queue.** A waiting list that always releases the lowest-priority item first. This is the one new building block; everything else carries over from last week.
3. **Implement greedy best-first search.** Take your search from last week and make the single change: swap the plain queue for the priority queue, and use the heuristic as the priority.
4. **Hand-trace it on a small graph.** Work through a few steps by hand (pop the lowest-h node, add its neighbours, repeat) so you can see *why* the search goes where it goes before you trust your code to do it.
5. **Compare guided against blind.** Run greedy and breadth-first on the same grid. Which finds the shorter route? Which explores fewer cells? Find a map where the fast search pays for its speed with a longer path.

Everything you need is the heuristic, the priority queue, and last week's search loop. There is no new mathematics beyond the two distance formulas above; only a smarter order in which to explore.

If you want to go further, short supplementary notes pick up exactly where greedy's flaw leaves off: a search that stays fast and *always returns the shortest path.*